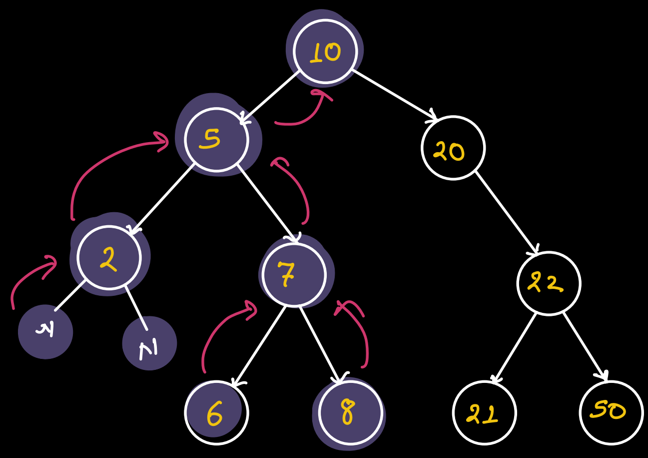
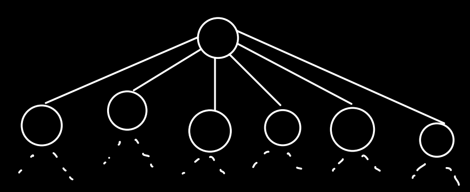
```
preorder(root){
    //Null check
    print(root.val)
    preorder(root.left);
    preorder(root.right);
}
```



10, 5, 2, 7, 6, 8, 20, 22, 21, 50

N-array Tree

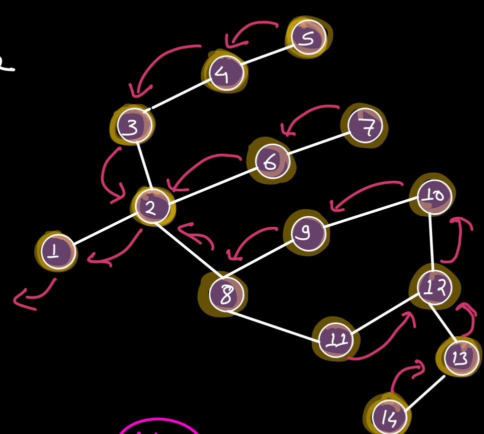
```
preorder(root){
    //Null check
    print(root.val)
    for(all nodes n in root.children){
        preorder(n);
    }
}
```



Non-tree Graph

```
DFS(v){
    mark v as visited;
    for(all nodes w in adj[v]){
        if(w is not visited){
            DFS(w);
        }
    }
}
```

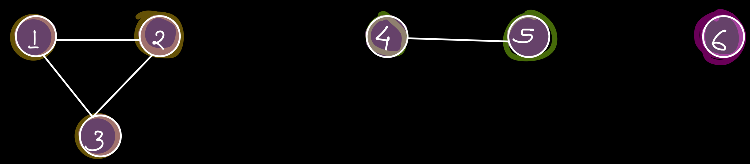
V ⟶ Source Node



HW

TC :

SC :



1 : 2, 3
2 : 1, 3
3 : 1, 2
4 : 5
5 : 4
6 :

```
for all vertices v in graph:
    if v is not visited:
        dfs(v);
```

Q Count no of connected components.

```
cc = 0;
for all vertices v in graph.
    if v is not visited:
        cc++;
        dfs(v);
```

Q. No. of Islands

Given a matrix of 1's & 0's.

0 → water

1 → land

Count the no. of island

↓

land surrounded by water in all 4 directions

$$\begin{matrix} 1 & 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 & 1 \\ 1 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 1 & 1 \end{matrix}$$

```
islands = 0;
for( i=0; i<N; i++){
    for (j=0; j<M; j++) {
        if (mat[i][j] == 1){
            dfs(mat, i, j);
            island++;
        }
    }
}
```

r : [-1, 0, 1, 0]

c : [0, 1, 0, -1]

```
dfs ( mat[][], Sr, Sc ){
    if (Sr<0 || Sr>=N || Sc<0 || Sc>=M ||
                                  mat[Sr][Sc] == 0)
        ret;
    mat [Sr][Sc] = 0;          // mark as visited
    for( i=0; i<4, i++){
        dfs(mat, Sr + r[i], Sc + c[i]);
    }
}
```

TC :

SC :

Q Given the map of conflicted area b/w two countries I & P.

"I" will take back all land that is completely surrounded by "I". Return the final state of the map.

I	I	P	I	I
I	P	P	I	I
I	I	I	I	I
I	P	I	I	P
I	P	P	I	P
I	I	I	I	P
I	I	I	I	I
I	I	I	P	I

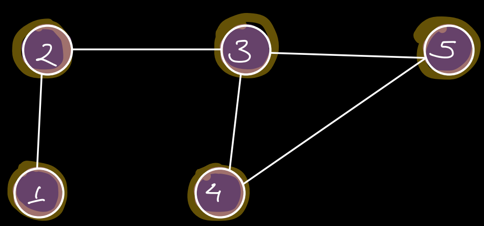
I	I	P	I	I
I	P	P	I	I
I	I	I	I	I
I	I	I	I	P
I	I	I	I	P
I	I	I	I	P
I	I	I	I	I
I	I	I	P	I

I	I	P	I	I
I	P	P	I	I
I	I	I	I	I
I	P	I	I	P
I	P	P	I	P
I	I	I	I	P
I	I	I	I	I
I	I	I	P	I

I	I	P	I	I
I	P	P	I	I
I	I	I	I	I
I	P	I	I	P
I	P	P	I	P
I	I	I	I	P
I	I	I	I	I
I	I	I	P	I

Q Given an undirected graph. Check if it is cyclic.

- Do BFS/DFS
- If you encounter a node that has been visited
  ⟶ cycle

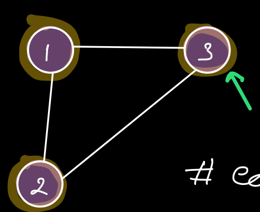


1 : 2
2 : 1, 3
3 : 2, 4, 5
4 : 3, 5
5 : 3, 4

Visited
2
1
3
4
5

No. of edges   N-K
5   >   4

Node		Parent
1	⟶	2
3	⟶	2
4	⟶	3
5	⟶	4



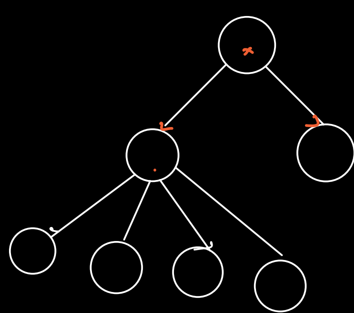
edges   N-K
3   >   2

1 : 2, 3
2 : 1, 3
3 : 1, 2

1
2
3

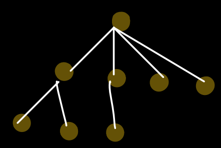
Connected Acyclic graph ⟶ Tree

N-1 edges



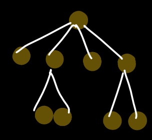
If there are 2 connected components (acyclic)

C_1 — N_1 nodes



$N_1 - 1$

C_2 — N_2 nodes



$N_2 - 1$

$N_1 + N_2 = N$

$\longrightarrow N_1 + N_2 - 2$

$\longrightarrow N - 2$

Three connected components

$N - 3$ edges

K connected components

\# edges $\overset{?}{<} = N - K$ (acyclic)

\# edges $> N - K$ (cyclic)